B2B CASE STUDY

Underwriting & Insurance Portal Platform

Developer → Senior Team Lead

Built from zero · legacy migration · ownership moved to a child company

Wonderland Inc. (anonymized) | Fintech · Underwriting | $100M+ risk portfolio

01 Summary

What this was, and what I owned

Helped build a B2B underwriting portal from zero — migrating a legacy monolith to a modern, container-based platform. I directly owned the configuration team and delivery flow while coordinating up to ~30 cross-functional contributors and the client.

3 → 10+
config team size

~30
cross-functional contributors

~4 mo
for 3 portals vs ~6 est.

20+
candidates interviewed

Headline: a three-portal request scoped at ~6 months shipped to production in ~4 months by restructuring it as a parent/child parallel build.

02 Context & problem

Migrate a legacy underwriting system, re-architect, and hand it to a child company

1. **Risk workflow** Digitize the insurance-record lifecycle end to end
2. **Documents** Generate & print policy and supporting documents
3. **Billing & reports** Handle bank billing and reporting
4. **Secure storage** Store data securely under compliance constraints
5. **Fast deploy** Repeatable, predictable deployment

CONSTRAINTS

Legacy system
monolith to migrate & re-platform

Compliance
regulated insurance data, secure storage

Ownership transfer
platform moving to a child company

03 Leadership scope

Being precise about what I ran, coordinated, and shaped

Direct ownership

- Config team
- Delivery flow
- Blockers
- Onboarding
- Config quality
- Estimates

Cross-functional coordination

- QA
- BA
- Dev
- DevOps
- PM
- Client stakeholders

Influence, not ownership

- Architecture
- Dev timeline
- Release planning
- UX / product decisions

Architecture sits under influence, not ownership — I advocated and shaped the calls; they weren't solely mine.

04 Technical approach

The stack, and what 'configuration' actually meant

STACK & TOOLING

Front-end	TypeScript / JavaScript
Back-end	C# / .NET · REST APIs
Data	Azure-managed data services · SQL
Delivery	Azure DevOps — Boards · Repos / PRs · Pipelines
Runtime	Kubernetes — deployment & security
Config & docs	Internal config tooling · Wiki

WHAT "CONFIGURATION" COVERED

A discipline in its own right — not setup after the fact.

- **Workflow states**
- **Underwriting rules**
- **Form fields**
- **Document templates**
- **User roles**
- **Permission sets**
- **Environment configs**
- **Business rules**
- **Billing/reporting maps**
- **Approval thresholds**

05 Architecture — C4 container view

An isolated security service governs access across every environment

Regular user | **Portal admin** | **Super admin**

Underwriting Portal Platform · runs on Kubernetes

Web SPA [TypeScript]	**API service** [C# / .NET]	**Document gen** [.NET]
Billing [.NET]	**Reporting** [.NET]	**Data store** [Azure SQL / storage]

ISOLATED

Admin & Security service

Roles · permissions · security settings

Multi-environment, with its own security schema

Every API request runs an authorization check here

Decision class: influence (advocated during the define phase). Payoff: manage access across environments without touching the full database — smaller blast radius.

06 Delivery process · Ideate → Deploy

The flow I owned end to end

1 — Stakeholder alignment

scope · estimates · budget · resources

→

2 — Team definition

ramp: BA+Architect → DevOps → Dev → QA

→

3 — Backlog & sprint planning

2 sprints + dependencies mapped

→

4 — Sprint loop 2–4 weeks

Refined → In Progress → QA → UAT → Done

→

5 — Deploy

runbook · smoke test · confirmation

BUILD ORDER

1 Core: schemas · workflow · attachments › 2 Key features › 3 Bugs & showstoppers › 4 Cosmetic UX/UI

Front-load the risky, dependency-heavy work; let low-risk polish be the thing that can safely slip.

07 User flow · risk-record lifecycle

From submission to a bound policy, with role-based branching

Assessment loops on "request more info" · decline closes the record early.

ROLE BRANCHING

Regular user — submit & view own records

Portal admin — approve referrals & high-value risks

Super admin — cross-portal config, security, environments

NORTH STAR

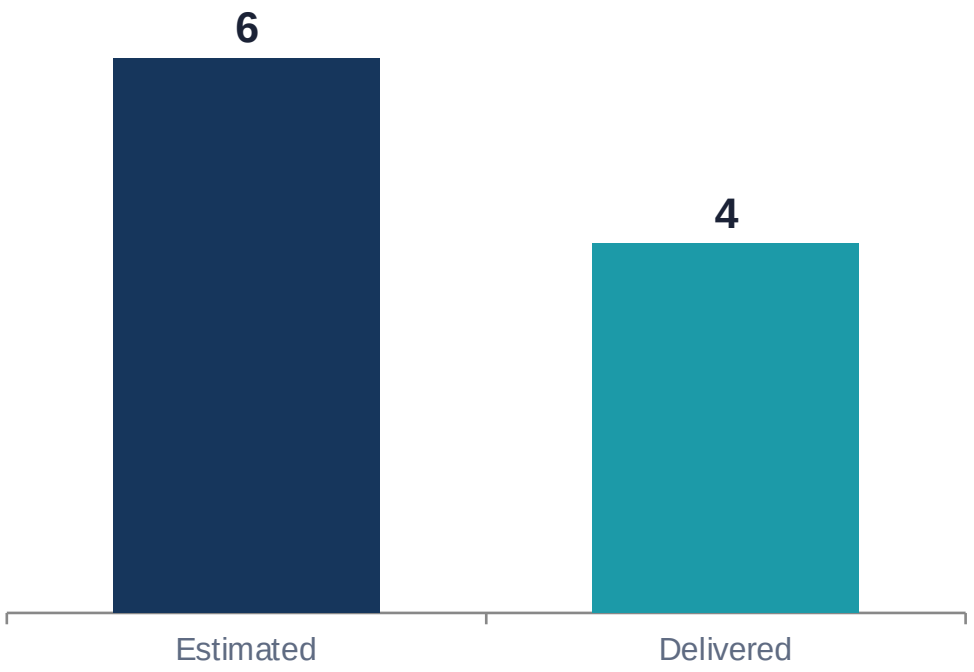
Time-to-first-bound-policy

Each step emits an event (risk_submitted → policy_bound) laddering up to activation & adoption — incl. % of config changes self-served by admins, no dev ticket.

08 Outcomes

Directional where exact figures are confidential

3 PORTALS — ESTIMATED vs DELIVERED



Release predictability ↑
runbook + branch-lock discipline

Dependency chaos ↓
merged 3 portals into one shared lineage

Repetitive onboarding Qs ↓
Wiki + interactive documentation

WHAT MATTERED MOST

Delivery the client and management could both predict — and a team that absorbed scope changes without thrash.

09 Stakeholder alignment

Holding three competing forces in balance

The hardest part was not the technical complexity — it was managing three competing forces at once.

CLIENT

Scope changes

adds & shifts requirements mid-cycle

MANAGEMENT

Timeline pressure

aggressive estimates; needs proof of progress

DELIVERY TEAM

Stable requirements

clear plan, fewer calls, fewer mid-cycle changes

OPERATING PRINCIPLE

Translate client ideas into options with dependencies and safety-margin estimates. Lock scope on client + management approval. Team runs ahead → pull items in; sprint overloaded → push to next. The end-of-sprint demo is management's recurring proof of progress.

10 People & cadence

How the team was built and how it ran

HIRING

20+ interviewed · multinational

1 · HR interview

2 · Technical — cognitive, reasoning, language, quiz, adaptive task

3 · Candidate review & calibration

ONBOARDING

- Basics docs & Wiki
- Responsibilities matrix
- AI-built interactive guide (sectioned, icon-based) for self-serve ramp
- **Goal: first PR in week one**

AGILE CADENCE

2-week sprints
tighter feedback than 4-week

Mid-sprint refinement
keeps planning short

Async daily + 15-min blocker call
fewer meetings

End-of-sprint demo
proof of progress

Retro every sprint
the key addition

REFLECTION

Spotting structural overlap between “separate” requests early — and consolidating the conversation around shared components — beat running them as independent tracks.

That single call turned ~6 months of sequential work into ~4 months of parallel delivery — and it was a leadership decision, not just a technical one.

Developer → Senior Team Lead · B2B Underwriting Portal Platform